

TransformOfShape

Summary

This function will apply the transformation matrix to the shape.

Usage

shape = *shape*:TransformOfShape(*matrix*)

Example

```
sh =sh:TransformOfShape(mat)
```

View LUT Plugin

ViewLut Plugins are used in the view to adjust color while being displayed. For example, a comp is processing true linear and displaying as sRGB in the view to match the monitor, View Luts do this conversion on the GPU using GLSL shaders. There are a number of common built-in ViewLuts, and Fuse View Lut Plugins can be developed to custom view looks.

ViewLut Creation

FuRegisterClass()

Summary

The FuRegisterClass function is executed when Fusion first loads the Fuse tool or ScriptViewShader. The arguments to this function provide Fusion with the information needed to properly present the tool for use by the artist. Fusion must be restarted before edits made to this function will take effect.

The FuRegisterClass function is required for all Fuse tools and ScriptViewShaders, and generally appears as the first few lines of the Fuse script.

Usage

FuRegisterClass(string name, enum ClassType, table attributes)

Returns

This event function does not return a value.

Arguments

name(string, required)

The name argument is a unique identifier that is used to identify the plugin to Fusion. It is also used as the base for the tool's default name. For example, the first instance a ScriptPlugin with the name 'Bob' would be added to the flow as Bob1.

ClassType(enum, required)

The ClassType is a predefined variable which identifies the type of Fuse for Fusion. For View LUT plugins, the value to use for the ClassType is CT_ViewLUTPlugin.

attributes (*table*, required)

The attributes table defines all the remaining options needed to define a Fusion tool. There are a wide variety of possible attributes, and not all are required. The following table lists the most common attributes, and their expected values. A more comprehensive list can be found at [FuRegisterClass Attributes](#).

Name	Description
REGS_Category	Required. A string value which sets the category a tool will appear in. For example, REGS_Category = "Script" will place the tool in the Scripts category of the tool menu. If the category does not exist, it will be created to hold the tool. Nested Categories can be defined using a \ character as a separator. For example, REGS_Category = "Script\\Color" will create a Color category under the Script category of the tool menu. Remember to use \\ instead of \ in a regular string, as \ is considered an escape character unless the [[]] raw string delimiters are used.
REGS_Name	Optional. Only needed if the ViewShader's displayed name is different to its unique ID.

Example

```
FuRegisterClass("GammaVSFuse", CT_ViewLUTPlugin, { -- ID must be unique
    REGS_Name = "Gamma ViewShader",
    REGS_Category = "ViewShaders",
})
```

ViewLut UI

Create()

Summary

The Create event function is executed when the ScriptViewShader is selected from the list of LUTs, once LUTs are enabled. Its job is to create any user controls, and to do any once-off setup for anything needed repeatedly later on. All objects created here are automatically destroyed when the ViewShader itself is destroyed.

The Create function does not require or use any arguments and does not return a value.

While all ScriptViewShaders must provide a Create event function, that function can be empty.

Adding controls is done with self.AddInput(), in the same fashion as with Fuse tools. Note that ViewShaders have no Output object, so no AddOutput() call is required.

Usage

Create()

Arguments

None

Example

```
function Create()
    InGamma = self:AddInput("Gamma", "Gamma", {
        LINKID_DataType = "Number",
        INPID_InputControl = "SliderControl",
        INP_Default = 1.0,
        ICD_MaxScale = 5.0,
    })
    InAlphaGamma = self:AddInput("Alpha Gamma", "AlphaGamma", {
        LINKID_DataType = "Number",
        INPID_InputControl = "SliderControl",
        INP_Default = 1.0,
        ICD_MaxScale = 5.0,
    })
end

-- This is called when the shader is created
-- img may be nil
function SetupShadeNode(group, req, img)
    -- pass group, name, params string, and shader source
    return ViewShadeNode(group, "GammaFuse", params, shader)
end
```

ViewLut Process

SetupShadeNode()

Summary

The SetupShadeNode event is called when Fusion needs to rebuild the display view's LUT shader chain, for example when the user changes the selected LUTs.

The fuse should construct a ViewShadeNode object, giving it a GLSL shader program in a string, then add any run-time parameters that will be passed to the shader, and return the object. A Request object containing the current control settings is passed in, along with the image being displayed (which may be nil), if required.

Usage

SetupShadeNode(group, request, image)

Arguments

group (*ViewShadeNodeGroup* object)

The group of shaders that this ViewShader will belong to. Needed for constructing a ViewShadeNode.

request (*Request* object)

A Request object containing the current control values.

image (*Image* object)

The source Image that will be given to the shader.

Returns

The function should return a freshly-created ViewShader object.

Example

```
-- Here's the GLSL shader itself:
-- params for the shader:
params =
[[
float Gamma          // enable switches (0 or 1), one per line
float Alphagamma
]]

shader =
[[
void ShadePixel(inout FuPixel f)
{
    // get source pixel
    EvalShadePixel(f);

    // apply Gamma
    vec4 gamma = vec4(Gamma, Gamma, Gamma, Alphagamma);

    f.Color = sign(f.Color) * pow(abs(f.Color), gamma);
}
]]

-- This is called when the shader is created
-- img may be nil
function SetupShadeNode(group, req, img)
    -- pass group, name, params string, and shader source
    return ViewShadeNode(group, "GammaFuse", params, shader)
end
```

SetupParams

Summary

The SetupParams event is called every time the screen is redrawn. This allows the fuse to pass any run-time parameters to the shader.

Parameters containing the current control values can be extracted from the passed Request. These values can be used to set the shader's run-time parameters in the passed ViewShadeNode with the SetParam() function.

Usage

SetupParams(request, vsnode, image)

Arguments

request (*Request* object)

A Request object containing the current control values.

vsnode (*ViewShadeNode* object)

The ViewShadeNode object created and returned by SetupShader()

image (*Image* object)

The source image that will be given to the shader

Returns

The function should return true, if successful. Returning false will cause the shader chain to be rebuilt, and FreeShader() then SetupShadeNode() will be called again. If SetupShadeNode() also returns nil, the ScriptViewShader will be bypassed completely until the user re-enables it.

Example

```
-- This is called every display refresh
-- img may be nil
function SetupParams(req, vsnode, img)

    -- retrieve control values
    local gamma      = InGamma:GetValue(req).Value
    local alphagamma = InAlphaGamma:GetValue(req).Value

    -- and setup the shader's parameter values
    vsnode:Set(1, gamma)
    vsnode:Set(2, alphagamma)

    return true
end
```

ViewLut Example

FuRegisterClass()

Summary

FuRegisterClass() registers your plugin within Fusion. It takes a unique ID string, a plugin class type enumerator, and a table of any extra attributes, like UI name. For viewshaders, the class type must be CT_ViewLUTPlugin.

Example

```
FuRegisterClass("GammaVSPuse", CT_ViewLUTPlugin, { -- ID must be unique
    REGS_Name = "Gamma ViewShader",
})
```

ViewShadeNode

A ViewShadeNode object encapsulates a GLSL shader program string, and a number of exposed, run-time parameters. It should be created by the SetupShadeNode() function, which is called when the shader chain is being built. The created ViewShadeNode is passed to SetupParams() every refresh of the display so that any run-time parameters can be set.

Constructor

ViewShadeNode Constructs a new ViewShadeNode object. This requires four arguments:

group	This is the ViewShadeNodeGroup passed to the SetupShadeNode() function.
name	This is an identifying name string.
params	This is a string containing a list of parameter types and names, one per line. Types can include the following: float vec2 vec3 vec4 mat4
shader	This is a string with the GLSL source for your shader program

Methods

Set	Sets the value of numeric parameters. Can be given a single number, or up to four values for a vector, or a Matrix4.
SetImage	Used to pass an Image to the shader as a texture.

Example

```
function SetupShadeNode(group, req, img)
    -- pass group, name, params string, and shader source
    return ViewShadeNode(group, "GammaFuse", params, shader)
end
```

The Shader String

The GLSL shader string used to construct a ViewShadeNode object consists of a function called ShadePixel(). This function takes a single argument, inout FuPixel f, which is used to return the view pixel being processed.

Methods

ShadePixel	Called to process each pixel.
EvalShadePixel	This is called to fetch the color values and location of the current pixel.

Example

This is an example of a simple gamma shader program:

```
shader =
[[
void ShadePixel(inout FuPixel f)
{
    // get source pixel
    EvalShadePixel(f);

    // apply Gamma
    vec4 gamma = vec4(Gamma, Gamma, Gamma, Alphagamma);

    f.Color = sign(f.Color) * pow(abs(f.Color), gamma);
}
]]
```

ShadePixel

Summary

The ShadePixel function is written in GLSL and executed by the GPU for every drawn pixel. A FuPixel is passed as the argument, and changes made to this FuPixel are passed on to the next shader in the chain when it calls EvalShadePixel().

The FuPixel structure has four members, all of which are four-component vectors:

- vec4 Color // source pixel color
- vec4 TexCoord0 // image pixel coords [0..w-1, 0..h-1]
- vec4 TexCoord1 // image normalised coords [0..1, 0..1]
- vec4 TexCoord2 // dest screen coords [0..scrw, scrh..0] (0 is top)

Usage

ShadePixel(inout *FuPixel* fpix)

fpix (*FuPixel*, required)

FuPixel struct used to receive changes to the current pixel

Returns

No explicit returns. This function modifies the FuPixel that is passed to it.

Example

```
void ShadePixel(inout FuPixel f)
{
    // get source pixel
    EvalShadePixel(f);

    // apply Gamma
    vec4 gamma = vec4(Gamma, Gamma, Gamma, Alphagamma);

    f.Color = sign(f.Color) * pow(abs(f.Color), gamma);
}
```